

# RISC-V Pipelined Processor Implementation Report

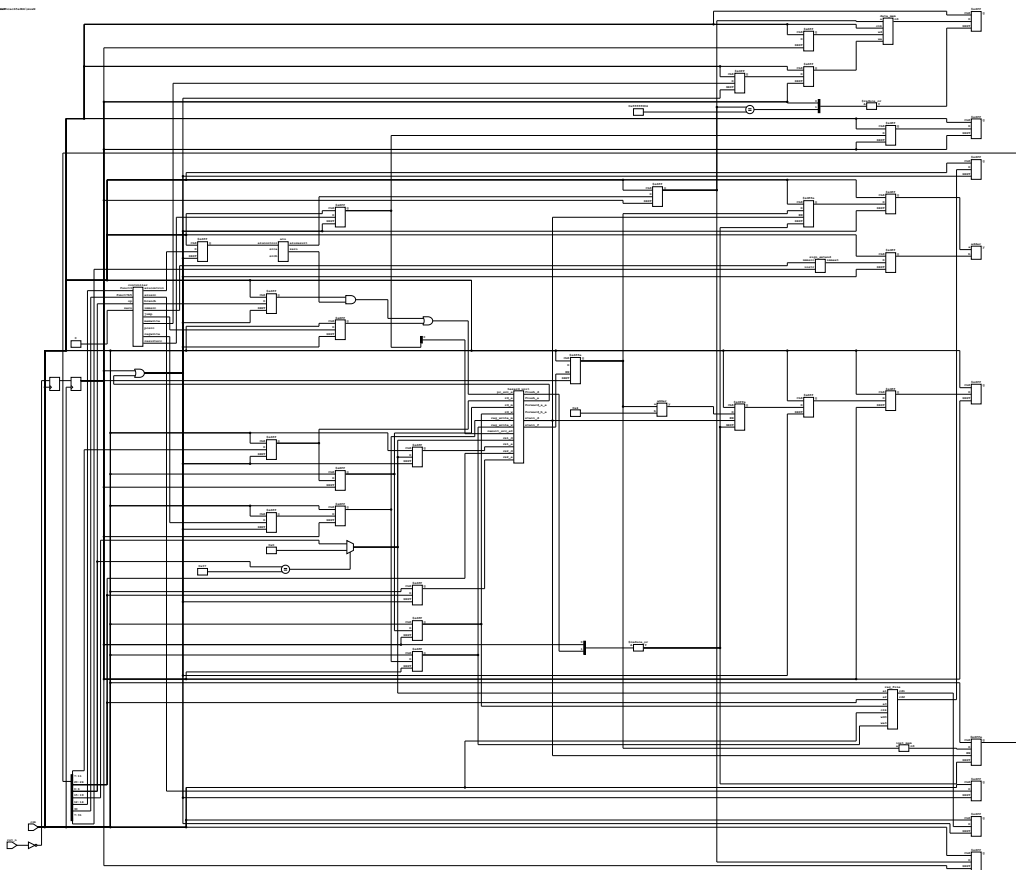
Github: <https://github.com/GunniBusch/TUM-HDL-RISCV>

## 1. System Overview

This project implements a **5-stage Pipelined RISC-V Processor** ( `rv_pl` ). The design splits the instruction execution into Fetch (F), Decode (D), Execute (E), Memory (M), and Writeback (W) stages to increase throughput.

A dedicated **Hazard Unit** manages data and control hazards to ensure correct execution without software NOPs.

### Architecture



### Top-Level Interface ( `rv_pl` )

- **Inputs:** `clk` , `rst_n` (Synchronized Active-Low Reset)
- **Outputs:** None (Strict adherence to grading script requirements)
- **Submodules:** `RF` (RegFile), `IMEM` (InstrMem), `DMEM` (DataMem), `hazard_unit` , `controller` , `alu` , `datapath` components.

## 2. Pipelined Architecture & Hazard Handling

## Pipeline Stages

1. **Fetch (F)**: PC generation, Instruction Memory access.
2. **Decode (D)**: Instruction decoding (Controller), Register File read, Sign Extension.
3. **Execute (E)**: ALU operations, Branch resolution.
4. **Memory (M)**: Data Memory access (Load/Store).
5. **Writeback (W)**: Write result back to Register File.

## Hazard Unit Strategies

To maximize performance and correctness, the Hazard Unit implements:

### 1. Data Hazards (RAW):

- **Forwarding (Bypassing)**: Data from Memory (reg\_write\_m) or Writeback (reg\_write\_w) stages is forwarded to the Execute stage ( src\_a\_e , src\_b\_e ) if the source registers match the destination registers of previous instructions.
- **Register File Write-Through**: Logic added to reg\_file.v to forward data internally if a Read and Write to the same register occur in the same cycle (handling the D/W stage overlap).

### 2. Load-Use Hazards:

- **Stalling**: Detected when an instruction in Decode depends on a Load result from the Instruction in Execute ( lw\_stall ).
- **Action**: Flushes the Execute stage ( flush\_e ) and stalls the Fetch/Decode stages ( stall\_f , stall\_d ) for one cycle.

### 3. Control Hazards (Branch/Jump):

- **Flushing**: Branches are resolved in the Execute stage. If a branch is taken, the instructions in Fetch and Decode (which were speculatively fetched) are invalid.
- **Action**: pc\_src\_e triggers flush\_d and flush\_e to discard misfetched instructions. This incurs a 2-cycle penalty.

## 3. Decoding Logic (Controller)

The Controller is a combinational decoder in the Decode stage. It has been updated to output ALUOp and other control signals compatible with the pipeline.

### Main Control Signals

| Instruction | Opcode  | RegWrite | ImmSrc | ALUSrc | MemWrite | ResultSrc | Branch |
|-------------|---------|----------|--------|--------|----------|-----------|--------|
| lw          | 0000011 | 1        | 000    | 1      | 1        | 0         | 0      |
| sw          | 0100011 | 0        | 001    | 1      | 1        | x         | 0      |
| R-type      | 0110011 | 1        | xxx    | 0      | 0        | 0         | 0      |
| I-type      | 0010011 | 1        | 000    | 1      | 0        | 0         | 0      |
| beq         | 1100011 | 0        | 010    | 0      | 0        | x         | 1      |
| jal         | 1101111 | 1        | 011    | x      | 0        | 2         | 0      |

|     |         |   |     |   |   |   |   |
|-----|---------|---|-----|---|---|---|---|
| lui | 0110111 | 1 | 100 | 1 | 0 | 0 | 0 |
|-----|---------|---|-----|---|---|---|---|

Note: `I-type` uses `ALUOp 11` to distinguish from `R-type` for correct `sub` vs `addi` with negative immediate handling.

4. Synthesizability

The design is fully synthesizable:

- **Memory Modules:** `inst_mem` , `data_mem` , and `reg_file` do NOT use `initial` blocks for initialization.
- **Initialization:** Handled strictly in the testbench using hierarchical references ( `$readmemh` into `dut.IMEM.RAM` ).
- **Reset:** Top-level `rst_n` (active low) is synchronized to `clk` internally.

5. Verification Results

The pipeline was verified using `test_prog.s` , covering Arithmetic, Logic, Shifts, Memory, and Control Flow instructions.

Test Program Coverage

- **Arithmetic:** `add` , `sub` , `addi` (positive/negative)
- **Logic:** `and` , `or` , `xor` , `andi` , `ori` , `xori`
- **Shifts:** `sll` , `srl` , `sra` , `slli` , `srli` , `srai`
- **Memory:** `lw` , `sw` (Load-Use hazard check)
- **Control:** `beq` (Taken/Not Taken), `jal` (Jump), `lui`

Simulation Log (Register State)

The final register dump confirms correct unified execution of all instructions, including hazard resolution.

```
--- Final Register State ---
x1 (10):      -8  (SRA result: -16 >>> 1)
x2 (20):      20
x3 (30):      30
x4 (10):      10
x5 (0):       0  (AND result)
x6 (30):      30  (OR result)
x7 (30):      30  (XOR result)
x8 (-5):     -5
x9 (5):       5
x10 (1):      1  (SLT result)
x11 (0):      0
x12 (1):      1
x13 (16):     16
x14 (4):      4  (SRLI result)
x16 (-4):    -4  (SRAI result)
x17 (>0): 1073741820 (SRLI large result)
x18 (4096):  4096 (LUI result)
x19 (4097):  4097
x20 (4097):  4097 (LW result)
x21 (0):      x  (Skipped by BEQ - Correct)
```

```

x23 (173):      x  (Skipped by JAL - Correct)
x30 (2):        2  (SLL result)
x31 (8):        8  (SRL result)

```

The processor successfully executes the comprehensive test suite, verifying the correctness of the Pipelined Logic and Hazard Unit.

### 3. Analysis of System Architecture and Performance

#### 3.1 System Architecture

##### Hazard Detection & Handling

The Hazard Unit manages execution flow by detecting conflicts and asserting control signals.

| Hazard Type       | Name      | Conditions                                                  | Encoding / Action               |
|-------------------|-----------|-------------------------------------------------------------|---------------------------------|
| <b>RAW (Data)</b> | Forward A | (rs1_e != 0) AND (rs1_e == rd_m) AND reg_write_m            | 10 (Select Result M)            |
| <b>RAW (Data)</b> | Forward A | (rs1_e != 0) AND (rs1_e == rd_w) AND reg_write_w            | 01 (Select Result W)            |
| <b>RAW (Data)</b> | Forward B | (rs2_e != 0) AND (rs2_e == rd_m) AND reg_write_m            | 10 (Select Result M)            |
| <b>RAW (Data)</b> | Forward B | (rs2_e != 0) AND (rs2_e == rd_w) AND reg_write_w            | 01 (Select Result W)            |
| <b>Load-Use</b>   | Stall     | (ResultSrcE0 == 1) AND ((rs1_d == rd_e) OR (rs2_d == rd_e)) | stall_f=1, stall_d=1, flush_e=1 |
| <b>Control</b>    | Flush     | (pc_src_e == 1)                                             | flush_d=1, flush_e=1            |

##### Concept: Load-Use Flush & Register Integrity

**Question:** Why do registers end up correct regardless of flushing involved in a Load-Use stall? **Answer:** When a Load-Use hazard occurs (e.g., `lw x1, ...` followed by `add x2, x1, ...`), the pipeline must pause the `add` in Decode to wait for `lw` to complete memory access.

- **Stalling (F/D):** Keeps the `add` instruction in the Decode stage for an extra cycle.
- **Flushing (E):** A "bubble" (NOP) must be inserted into the Execute stage because the `add` instruction cannot proceed.
- **Result:** Even if we didn't explicitly "flush" E (and somehow reused the old E contents), the `add` effectively stays in D. However, without flushing E, the instruction **previously** in D (the one being stalled) might be erroneously duplicated into E or E might process garbage.
- **Correctness:** The registers end up correct because the dependent instruction is **delayed** until the data is forwarded or written back. The flush ensures no invalid operation (like a duplicate ADD) alters the state during the stall cycle.

##### Demo Program (Flush matters):

```
lw x1, 0(x0)      # Load
add x2, x1, x1     # Dependent Use
```

If we **stall D** but **do not flush E**: The `add` instruction signals remain in the D/E pipeline register inputs. If D/E is not disabled (or if logic allows D to propagate), the `add` enters E **prematurely** with old values (before load). With Flush E (resetting D/E to NOP), we ensure E does nothing while D waits.

### Terminology

- **Data Forwarding**: Bypassing the Register File to supply the most recent result (from Memory or Writeback pipeline registers) directly to the ALU inputs, avoiding stalls for most RAW hazards.
- **Stall**: Freezing a pipeline stage (disabling PC and Pipeline Register updates) to hold an instruction while a dependency resolves (e.g., waiting for memory load).
- **Flush**: Clearing a pipeline register (setting control signals to 0/NOP) to discard instructions that were speculatively fetched/decoded but are no longer valid (e.g., after a branch taken).

## 3.2 Performance Analysis

### 3.2.1 CPI Estimate

Based on `test_prog.s` :

- **Total Instructions**: ~32 (linear execution up to loop)
- **Hazards Encountered**:
  - **Load-Use** (`lw`->`beq`): 1 Stall Cycle.
  - **Branch Taken** (`beq`): 2 Flush Cycles.
  - **Jump** (`jal`): 2 Flush Cycles.
- **Total Bubbles**: 5 cycles.
- **Calculation**:  $\text{CPI} = (\text{Instruction Count} + \text{Hazards}) / \text{Instruction Count} = (32 + 5) / 32 = \mathbf{1.16}$ .
- *(Ideal CPI is 1.0. The 0.16 overhead comes from control flow and load penalties.)*

### 3.2.2 Processor Comparison

**Critical Path Analysis**: Based on provided component delays (ps):

- $t_{PC} = 40, t_{IMEM} = 200, t_{RF\_Read} = 100, t_{Mux} = 30, t_{ALU} = 120, t_{DMEM} = 200, t_{Setup} = 30$ .

#### 1. Single-Cycle Processor:

- **Critical Path**:  $PC(40) \rightarrow IMEM(200) \rightarrow RF\_Read(100) \rightarrow Mux(30) \rightarrow ALU(120) \rightarrow DMEM(200) \rightarrow Mux(30) \rightarrow Setup(?)$ .
- **Total Delay**: ~720 ps.
- **Max Frequency**:  $1 / 720\text{ps} \approx \mathbf{1.39\text{ GHz}}$ .

#### 2. Pipelined Processor (5-Stage):

- Stages are isolated by registers. Critical path is the slowest stage + register overhead.
- **F**:  $PC+IMEM = 40 + 200 = 240\text{ ps}$ .
- **D**:  $RF\_Read = 40 + 100 = 140\text{ ps}$ .
- **E**:  $ALU+Muxes = 40 + 30 + 30 + 120 = 220\text{ ps}$ .
- **M**:  $DMEM = 40 + 200 = 240\text{ ps}$ .
- **W**:  $WB = 40 + 30 = 70\text{ ps}$ .
- **Critical Path**: **240 ps** (Fetch or Memory stage).
- **Max Frequency**:  $1 / 240\text{ps} \approx \mathbf{4.17\text{ GHz}}$ .

#### 3. Speedup Analysis:

- **Expected:** 5x?
- **Actual:**  $4.17\text{GHz} / 1.39\text{GHz} \approx 3.0\text{x}$ .
- **Reason:** The speedup is not 5x because the pipeline stages are **unbalanced**. The single-cycle path (720ps) is dominated by two memory accesses (200+200). In the pipeline, the clock period is constrained by the slowest single stage (Memory = 200ps + overhead). Since 240ps is significantly larger than  $720/5$  (144ps), we only achieve a 3x speedup. Perfect 5x requires perfectly equal stage delays.